DLP Full Stack Home Assignment – Simplified DLP Platform

Overview

This assignment focuses on building a simple DLP backend application that allows managing inspection rules,

grouping them into policies, and scanning text input to detect sensitive content.

The goal is to demonstrate clear backend design, correct logic implementation, and a basic understanding of API and data modeling.

Objective

Implement a backend application that exposes HTTP APIs to:

Manage Data Types (DTs)

Manage Data Sets (DSs)

Scan input text and return inspection results

Architecture

The solution should be implemented as a single backend application, structured into three logical API modules:

/api/datatypes – Data Types management

/api/datasets – Data Sets management

/api/scan – Scan endpoint

Core Requirements

A single backend service

A single database

CRUD operations for Data Types and Data Sets

A synchronous scan endpoint returning immediate results

Data Models

Data Types (DTs)

Attributes:

id: UUID (generated internally)

name: STRING

description: STRING

type: ENUM (keywords only)

content: STRING ARRAY

threshold: NUMBER

At the moment, only keyword-based inspection is required.

Data Sets (DSs)

Attributes:

id: UUID (generated internally)

name: STRING

data_type_ids: ARRAY of DT UUIDs

Scan Flow

Receive scan request (input text, Data Set ID)

Load Data Set

Load referenced Data Types

Execute scan logic

Return result

Matching Logic

Each Data Type evaluated independently

Match if keyword count ≥ threshold

Whole-word keyword matching

OR logic across Data Types

Scan Result Examples

Match:

{
"status": "match",
"detected_objects": [
{
"id": "dt-uuid",
"name": "Sensitive Keywords",
"match_count": 3
}
]
}

Not matched:

{
"status": "not matched"
}

Optional (Nice to Have)

Docker or docker-compose setup

Basic input validation

Clear project structure

README explaining how to run the application